Module 2: Advanced Front-end Development with React

Topic 2 Objective Key Result

Objectives:

- Achieve a comprehensive understanding of front-end development using the **React framework**, effectively leveraging its core features and popular libraries.
- Master advanced techniques for building **scalable**, **maintainable**, and **interactive** React applications.

Key Results:

- Proficiency in creating and structuring **complex React projects**, utilizing modern development practices.
- Ability to integrate and effectively utilize a range of **libraries** for state management, routing, UI, and more within React projects.
- Capability to develop applications that interact with **APIs**, manage application **state efficiently**, and implement user-friendly **routing**.

Topic 2.1: Diving Deeper into React ⚛️

React stands as a cornerstone in modern website development, particularly for crafting dynamic and responsive user interfaces. It elegantly merges JavaScript's power with HTML's structure through **JSX**, a syntax extension. At its heart, React operates on a **Virtual DOM**, an in-memory representation of the actual DOM. This allows React to efficiently update only the necessary parts[1] of the UI, leading to significant performance gains. Development in React revolves around building **components**, which are reusable, self-contained pieces of UI. While traditionally, React utilized Object-Oriented principles through **class components**, the paradigm has shifted towards **functional components** powered by **Hooks**.
React projects predominantly use .js and .jsx files. While .js can contain React code, .jsx explicitly signals the presence of JSX syntax, often preferred for components to enhance readability and tooling support.

Core Concepts Revisited:

- **Components:** The building blocks. We primarily use functional components with arrow functions.
- **JSX:** Allows writing HTML-like structures within JavaScript, offering a more intuitive way to define UI.
- **Props (Properties):** How components receive data from their parents, enabling data flow (typically top-down).

- **State:** Data that a component manages internally and can change over time, causing re-renders.

Introducing Hooks:

Hooks are functions that let you "hook into" React state and lifecycle features from functional components.

- **useState:** Allows adding state to functional components.

```
import React, { useState } from 'react';

const Counter = () => {
  const [count, setCount] = useState(0); // [stateValue, updaterFunction]

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>Click me</button>
    </div>
  );
};
```

- **useEffect:** Lets you[2] perform **side effects** (data fetching, subscriptions, manual DOM changes). It runs after every render by default but can be controlled with a dependency array.

```
import React, { useState, useEffect } from 'react';

const DataFetcher = () => {
  const [data, setData] = useState(null);

  useEffect(() => {
    fetch('https://api.example.com/data')
      .then(response => response.json())
      .then(result => setData(result));

    return () => console.log("Component unmounted"); // Optional cleanup
  }, []); // Empty array = runs once on mount

  return <div>{data ? <p>Data loaded!</p> : <p>Loading...</p>}</div>;
```

```
};
```

- **useContext:** Passes data through the component tree without manual prop drilling. https://medium.com/zestgeek/mastering-reacts-usecontext-hook-simplifying-state-management-65894e6dc431

Conditional Rendering & Lists:
Render UI based on conditions (if or ternary operator) and map arrays to lists, remembering to use the key prop.

```
const ItemList = ({ items }) => {
  return (
    <ul>
      {items.length > 0 && items.map(item => (
        <li key={item.id}>{item.name}</li>
      ))}
    </ul>
  );
};
```

Exporting and Importing:
Use export default or named exports (export const ...) to share components between files.
Resources:

- React Official Documentation
- Hooks at a Glance
- Thinking in React

Topic 2.2: Expanding Your Toolkit: Packages, Libraries, and Routing 🛠️

Real-world React apps need more than just the core library. We use external packages and libraries for enhanced functionality.
Packages and Libraries:

- **Libraries:** Offer specific functions (e.g., react-icons, axios).
- **Packages:** Bundles managed via npm/yarn.

UI Frameworks/Libraries:
Provide pre-built components for faster, consistent UI development.

- **Material UI (MUI):** Google's Material Design components.
- **Chakra UI:** Focuses on accessibility and developer experience.
- **React Bootstrap:** Adapts Bootstrap for React.

- **Tailwind CSS:** A utility-first CSS framework for building custom designs directly within your markup, without writing traditional CSS.
- **shadcn/ui:** A collection of reusable, accessible, and highly composable UI components built on Radix UI and Tailwind CSS, which you add directly to your codebase via a CLI.

Routing with react-router-dom:
Manages navigation in Single Page Applications (SPAs) without full page reloads.

```
import React from 'react';
import { BrowserRouter as Router, Routes, Route, Link } from 'react-router-dom';
import Home from './Home';
import About from './About';

const App = () => {
  return (
    <Router>
      <nav>
        <Link to="/">Home</Link> | <Link to="/about">About</Link>
      </nav>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </Router>
  );
};

export default App;
```

State Management Libraries:
For complex apps, libraries like **Redux** or **Zustand** offer robust global state management.

Resources:
- React Router DOM
- Material UI
- Chakra UI
- React Bootstrap
- Axios (for HTTP requests)

Advanced Task 🌟

Project: Interactive Content Dashboard
Objective: Build a multi-page React application using static data, allowing users to browse, search, view detailed data, and add to favorites.
Theme Ideas: Movie Dashboard, News Dashboard, E-commerce Lister, Art Dashboard.
Requirements:

1. **Setup:** Create a React project (Vite or Create React App).
2. **Routing:** Implement at least three routes (Home/Listing, Details, Favorites) using react-router-dom.
3. **Components:** Use reusable components (ItemList, ItemCard, etc.).
4. **Styling:** Use a UI library (Tailwind CSS & shadcn/ui) for a consistent, responsive design.
5. **State:** Use useState for local state. Implement "Favorites" using useContext or a simple library (persist with localStorage).
6. **Interactivity:** Add search/filtering. Implement "Favorite" toggling.
7. **Deployment (Optional):** Deploy to Vercel, Netlify, or GitHub Pages.
8. **Others (Optional):** Implement CRUD for dynamic data.

Sources

1. https://codersjoy.com/category/web-development/feed
2. https://github.com/samuelblack11/CheatSheets
3. https://github.com/mosaddekalimithu/react-road-map
4. https://github.com/TenexDesigns/Next-JS
5. https://hackmd.io/@AnzenKodo/IP-UT1-QB?utm_source=preview-mode&utm_medium=rec
6. https://github.com/Adiljutt264/React-life-cycle-p2